

AFRICA INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND EMERGING TECHNOLOGIES

DAI 022 — MACHINE LEARNING
FINAL EXAM PROJECT

Predicting Student Academic Performance Using Machine Learning: A Classification Approach

Student Name: [Your Name]

Registration Number: [Your Reg No.]

Programme: Diploma in AI and Cybersecurity

Lecturer: [Lecturer's Name]

Date of Submission: [Date]

1. Introduction

Many students struggle academically, and this often only becomes visible after final exam results are released, by which point it is too late to intervene. Teachers and institutions typically discover a student is at risk after the grade is already recorded, even though warning signs such as declining attendance, reduced study time, and poor sleep patterns are often present much earlier.

Manually tracking attendance, study habits, sleep, and assignment performance across hundreds of students is not practical for a teacher to do by hand. This is exactly the kind of pattern-recognition problem that machine learning is well suited to: identifying combinations of behaviours that are associated with particular academic outcomes.

This project applies machine learning classification techniques to predict a student's performance category (Fail, Pass, or Distinction) based on their study habits, attendance, sleep, assignment performance, and past academic history. The aim is to build an early-warning system that could help teachers, academic advisors, and students themselves identify risk before final exams occur, and to compare several supervised and unsupervised techniques to understand which best captures meaningful patterns in student behaviour.

2. Project Objectives

General Objective

To apply basic machine learning techniques to analyse student data, build predictive models, evaluate their performance, and interpret results to support early identification of academic risk.

Specific Objectives

- Collect and describe a suitable dataset containing student behavioural and academic performance data.
- Clean and prepare the dataset for machine learning, including handling missing values, duplicates, and categorical encoding.
- Perform exploratory data analysis to identify patterns and relationships between student behaviour and academic performance.
- Train and compare supervised classification models (Logistic Regression, Decision Tree, and a Neural Network) to predict student performance categories.
- Apply K-Means clustering to uncover natural behavioural groupings among students.
- Evaluate model performance using accuracy, precision, recall, and F1-score.
- Interpret the results and provide practical, ethically-grounded recommendations for real-world use.

3. Dataset Description

The dataset used in this project was synthetically generated in Python to represent 500 students, with feature relationships deliberately built to reflect realistic academic patterns (e.g., higher attendance and

study hours are associated with higher exam scores, with added random variation to reflect real-world unpredictability). A synthetic dataset was used after a suitable public dataset could not be located; the assignment brief explicitly permits creating a dataset where appropriate.

Column	Description
student_id	Unique identifier for each student
hours_studied	Average daily study hours (0-10)
attendance_percent	Class attendance percentage (40-100%)
sleep_hours	Average nightly sleep hours (4-9)
assignment_score	Average assignment score (0-100)
previous_scores	Average score in past exams (30-100)
extracurricular	Participation in extracurricular activities (Yes/No)
exam_score	Final exam score (0-100) - used to derive the target
performance_category	Target variable: Fail / Pass / Distinction

Rows: 501 (500 students, with 1 duplicate row intentionally introduced for the data preparation exercise) | **Columns:** 9 | **Target variable:** performance_category | **Problem type:** Multi-class classification (Fail / Pass / Distinction)

[INSERT SCREENSHOT HERE: df.head() output showing the first 5 rows of the dataset]

4. Machine Learning Workflow

This project follows the standard machine learning workflow: problem identification, data collection, data cleaning, data exploration, feature selection, train-test splitting, model training, model evaluation, result interpretation, and ethical reflection. Each stage is documented in the sections that follow, showing that machine learning involves not only writing code but also understanding the problem and critically interpreting results.

5. Data Preparation

Before modelling, the dataset was checked for missing values and duplicate rows.

[INSERT SCREENSHOT HERE: df.info() and/or df.describe() output]

5.1 Missing Value Check

Using `df.isnull().sum()`, 7 missing values were found in `sleep_hours` and 8 in `assignment_score` (15 total, out of 501 rows, roughly 3% of the data).

[INSERT SCREENSHOT HERE: df.isnull().sum() output]

5.2 Cleaning: Duplicates and Missing Values

One duplicate row was identified using `df.duplicated().sum()` and removed with `df.drop_duplicates()`. Missing values in `sleep_hours` and `assignment_score` were filled using the column median rather than the mean, since the median is less sensitive to outliers and better represents a "typical" student. Dropping the affected rows entirely was considered but rejected, since with only 500 students, discarding rows would lose potentially useful information from other columns for a relatively small amount of missing data.

[INSERT SCREENSHOT HERE: Data cleaning code (drop_duplicates and fillna) and confirmation output]

5.3 Encoding Categorical Variables

The extracurricular column (Yes/No) was converted to numeric values (1 = Yes, 0 = No) using a manual mapping, since it is a simple binary category with an obvious numeric meaning.

5.4 Feature Selection and Train-Test Split

The feature set (X) excludes `student_id` (a non-predictive identifier) and `exam_score`. Excluding `exam_score` is critical: since `performance_category` was directly derived from `exam_score`, including it as a feature would cause data leakage, letting the model "see the answer" rather than learn genuine patterns. The data was split into 80% training and 20% testing sets using `train_test_split()`, with stratification to preserve the proportion of each performance category in both sets.

[INSERT SCREENSHOT HERE: Train-test split code and resulting shapes (X_train, X_test, y_train, y_test)]

6. Exploratory Data Analysis

Three visualisations were produced to understand the dataset before modelling.

6.1 Distribution of Exam Scores

A histogram with a kernel density overlay showed a roughly bell-shaped distribution of exam scores, with most students clustered around the average and fewer students at the very low and very high ends.

[INSERT SCREENSHOT HERE: Histogram of exam_score distribution]

6.2 Correlation Heatmap

A correlation heatmap of the numeric features showed that `hours_studied`, `attendance_percent`, and `assignment_score` had the strongest positive correlation with `exam_score`, indicating these were likely to be the most predictive features going into modelling.

[INSERT SCREENSHOT HERE: Correlation heatmap of student features]

6.3 Attendance vs Exam Score by Performance Category

A scatter plot of attendance against exam score, coloured by performance category, showed visibly distinct groupings: Fail students clustered toward lower attendance and lower scores, Distinction students clustered toward higher attendance and higher scores, with Pass students in between. This

suggested that a classification model would likely be able to learn meaningful separation between the categories.

[INSERT SCREENSHOT HERE: Scatter plot of attendance_percent vs exam_score, coloured by performance_category]

7. Supervised Learning Model: Logistic Regression

Logistic Regression was selected as the primary supervised classification model. Numeric features were scaled using `StandardScaler` before training, since Logistic Regression is sensitive to differences in feature magnitude (e.g., `attendance_percent` ranges 40-100 while `sleep_hours` ranges 4-9; without scaling, larger-magnitude features could dominate purely due to their scale, not their actual importance).

[INSERT SCREENSHOT HERE: Logistic Regression training code (model.fit)]

The trained model was then used to predict performance categories on the unseen test set.

[INSERT SCREENSHOT HERE: Logistic Regression prediction code and sample output]

8. Decision Tree Model

A Decision Tree Classifier was trained as a second supervised model, chosen for its interpretability, since it is possible to visually trace exactly how it arrives at a prediction. Decision trees do not require feature scaling, since they split on raw thresholds (e.g. "`attendance_percent <= 78.5`") rather than distance calculations.

The tree depth was limited to `max_depth=2` (a simplified version) for interpretability, and `max_depth=3` for the main model. Limiting depth is a form of **pre-pruning**: an unrestricted tree would keep splitting until it perfectly memorises the training data, a form of **overfitting** where the model performs well on training data but poorly on new, unseen data. By restricting depth, the tree is forced to retain only the strongest, most generalisable splits.

Each node in the tree shows: the splitting question (e.g. a threshold on `hours_studied`), the Gini value (a measure of how mixed the classes are in that node, where 0 means perfectly pure), the number of training samples reaching that node, the class breakdown of those samples, and the majority class prediction for that node.

[INSERT SCREENSHOT HERE: Decision tree training code]

[INSERT SCREENSHOT HERE: Decision tree visualisation (plot_tree output)]

9. Unsupervised Learning: K-Means Clustering

K-Means clustering was applied using five behavioural features: `hours_studied`, `attendance_percent`, `sleep_hours`, `assignment_score`, and `previous_scores`. These features were chosen because they represent a student's overall effort and habits, while `performance_category` and `exam_score` were deliberately excluded so that clustering was based purely on behaviour, not on the outcome being

predicted. Features were scaled before clustering, since K-Means relies on distance calculations between data points, and unscaled features with larger ranges would otherwise dominate the distance metric.

Three clusters were chosen to mirror the three performance categories, allowing a comparison between behaviour-based groupings and actual outcomes.

[INSERT SCREENSHOT HERE: K-Means clustering code and cluster value_counts() output]

9.1 Cluster Interpretation

Cluster	Size	Profile	Interpretation
0	167	High attendance (85.3%), high previous scores (76.0)	Consistent, high-attendance performers
1	138	High study hours (6.0), high assignment scores (77.5), but lower attendance (70.4%) and previous scores (58.6)	High effort students with attendance gaps
2	195	Lowest study hours (3.9), highest sleep (7.5), decent attendance (82.6%)	Low effort, well-rested students

When cross-tabulated against the real performance_category, Cluster 0 skewed toward Pass and Distinction with the fewest Fails, Cluster 2 skewed heavily toward Fail and Pass with the fewest Distinctions, and Cluster 1 was the most evenly split across all three categories. This shows that behavioural patterns alone, with no access to performance labels, carry real predictive signal: consistent attenders tended toward better outcomes, low-effort students tended toward worse outcomes, and high-effort students with attendance gaps produced the most mixed results, suggesting effort can partially, but not consistently, offset lower attendance.

[INSERT SCREENSHOT HERE: Cluster visualisation: scatter plot of clusters vs actual performance categories (side by side)]

10. Introduction to Neural Networks

A simple neural network (Multi-Layer Perceptron) was implemented using scikit-learn's MLPClassifier, with one hidden layer of 10 neurons. A small network was chosen given the limited size of the dataset (approximately 400 training rows) and the small number of features, to avoid unnecessary complexity and overfitting.

- **Input layer:** represents the six input features (hours_studied, attendance_percent, sleep_hours, assignment_score, previous_scores, extracurricular). Each input neuron corresponds to one feature value for a given student.
- **Hidden layer (10 neurons):** combines the inputs in different weighted ways and passes them through a non-linear activation function, allowing the network to learn combinations of features that a simpler linear model may not capture.
- **Output layer:** produces the final predicted category (Fail, Pass, or Distinction) based on which output neuron is most strongly activated.

[INSERT SCREENSHOT HERE: Neural network training code (MLPClassifier.fit) and prediction output]

11. Model Evaluation and Comparison

All three classification models were evaluated on the same held-out test set using accuracy, weighted precision, weighted recall, and weighted F1-score. Weighted averaging was used since the dataset has three classes of slightly different sizes.

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.660	0.654	0.660	0.655
Decision Tree	0.480	0.505	0.480	0.471
Neural Network	0.610	0.600	0.610	0.602

[INSERT SCREENSHOT HERE: Evaluation output: accuracy, classification report, and confusion matrix for each model]

Accuracy measures the overall proportion of correct predictions. **Precision** measures, of all students predicted into a category, how many actually belonged there (false-alarm rate). **Recall** measures, of all students who actually belonged to a category, how many the model correctly identified (miss rate). **F1-score** balances precision and recall into a single number.

Logistic Regression performed best (66% accuracy), suggesting the relationship between student behaviour and exam performance in this dataset is largely linear and additive. The **Neural Network** performed second best (61%), likely limited by the relatively small training set (~400 rows), which is not enough data for a neural network to significantly outperform simpler models. The **Decision Tree performed weakest** (48%), a direct trade-off of limiting its depth to prevent overfitting and preserve interpretability; a deeper, unrestricted tree might score higher on this specific test set but would generalise worse to new students and be harder to explain.

None of the models exceeded 70% accuracy, which is expected and appropriate given that performance_category was generated with genuine random noise built in, meaning no model could achieve perfect prediction. This reflects the real unpredictability of human academic performance rather than a modelling error.

12. Data Visualization of Results

Confusion matrices for all three classification models were plotted side by side to visually compare where each model succeeds and where it makes mistakes. In a confusion matrix, rows represent the true category and columns represent the predicted category; the diagonal cells represent correct predictions, while off-diagonal cells represent misclassifications. Errors were more common between adjacent categories (e.g. Pass mistaken for Distinction) than between distant categories (e.g. Fail mistaken for Distinction), which is expected, since borderline students genuinely share overlapping characteristics.

[INSERT SCREENSHOT HERE: Confusion matrix heatmaps for all three models, side by side]

13. Challenges Encountered

Missing Values

The dataset contained missing values in `sleep_hours` and `assignment_score`. This was resolved by imputing the median of each column, preserving the affected rows while minimising the influence of outliers on the replacement value.

Class Imbalance in Category Design

An early version of `performance_category` used fixed score thresholds (e.g. below 50 = Fail), which produced a heavily imbalanced dataset (401 Pass, 32 Fail). This was resolved by switching to percentile-based thresholds, producing a more balanced and evaluable set of categories.

Data Leakage Risk

Since `performance_category` was directly derived from `exam_score`, an early design risk was accidentally including `exam_score` as an input feature, which would have let the model "see the answer" rather than learn genuine patterns. This was addressed by explicitly excluding `exam_score` from the feature set used for training.

Lower Decision Tree Accuracy

The Decision Tree achieved noticeably lower accuracy (48%) than the other two models. This is a direct trade-off of limiting `max_depth` to avoid overfitting; a deeper tree may have scored higher on this specific test set but would generalise worse to new data and lose interpretability.

Visualising Multi-Dimensional Clusters

Since K-Means used five features simultaneously, the clusters could not be directly visualised in a single 2D plot. This was addressed by selecting the two features with the clearest separation between cluster averages for visualisation, while acknowledging that the clustering itself considered all five features.

14. Ethical Discussion

Risk of Labelling and Self-Fulfilling Prophecies

Predicting that a student is likely to "Fail" could unintentionally influence how teachers, parents, or the students themselves perceive their ability. A student labelled at-risk early on could face lowered expectations, turning the prediction into a self-fulfilling prophecy rather than a helpful early warning.

Data Privacy

Student data such as attendance, sleep patterns, and academic history is sensitive. Any real deployment of a system like this would need informed consent, secure storage, and use restricted to its intended purpose of supporting students rather than punitive action.

Bias and Fairness

This project used a synthetic dataset with controlled, transparent relationships between features. A real-world dataset could reflect existing inequalities; for example, students from under-resourced backgrounds might show lower attendance due to circumstances outside their control (transport, home responsibilities, health) rather than lack of effort. A model trained on such data could unfairly penalise students for systemic issues rather than individual choices.

Over-Reliance on Automated Predictions

Machine learning predictions should support human judgement, not replace it. A teacher who understands a student's personal circumstances may recognise context a model cannot see. Such a system should function as a decision-support tool alongside teacher input, not as the sole determinant of a student's academic path.

Transparency

Students and parents affected by such predictions should be informed about how the system works and have the ability to question or appeal predictions that do not reflect their real circumstances.

15. Conclusion and Recommendations

Conclusion

This project applied machine learning techniques to predict student academic performance based on study habits, attendance, sleep, assignment scores, and past academic history. A dataset of 500 students was cleaned, explored, and used to train three classification models (Logistic Regression, Decision Tree, and a Neural Network), alongside K-Means clustering to uncover natural behavioural groupings among students.

Logistic Regression achieved the best performance among the classification models, with 66% accuracy, followed by the Neural Network (61%) and the Decision Tree (48%). This suggests the relationship between student behaviour and academic outcomes in this dataset is largely linear and additive, favouring simpler models over more complex ones. The Decision Tree, while less accurate, offered the clearest interpretability, directly showing which features (attendance and study hours) most influenced predictions.

K-Means clustering, despite having no access to actual performance labels, uncovered three behavioural groups that showed a meaningful relationship with real outcomes: consistent attendees leaned toward better results, low-effort students leaned toward Fail, and hard workers with attendance gaps showed the most mixed outcomes. This confirms that behavioural patterns alone carry real predictive signal, even without labelled data.

Overall, the project demonstrates that machine learning can meaningfully support early identification of at-risk students, though no model achieved particularly high accuracy, reflecting the genuine complexity and unpredictability of human academic performance. In practice, this system could be integrated into a school's existing student information system to flag at-risk students each term, allowing academic advisors to prioritise outreach and support before final exams rather than after grades are released. It could also help students track how their own habits, such as attendance and study time, relate to their likely outcomes, encouraging proactive changes.

Recommendations

- **Use predictions as a support tool, not a final verdict.** Teachers and academic advisors should treat model outputs as an early-warning signal to prompt further conversation with a student, not as an automatic determination of their ability.
- **Collect richer, real-world data for future work.** This project relied on a synthetic dataset with five behavioural features. A production system would benefit from richer, real data, such as subject-specific performance, socioeconomic context, or engagement with learning resources.
- **Regularly re-evaluate the model for bias.** If deployed with real student data, the model should be periodically audited to check whether it disproportionately flags students from particular backgrounds.
- **Combine multiple models rather than relying on one.** Since Logistic Regression, the Decision Tree, and the Neural Network each captured slightly different patterns, an ensemble approach or a simple majority vote across models could improve robustness in a real deployment.

References

- Dataset: synthetically generated for this project using Python (NumPy, pandas), designed to reflect realistic relationships between student behaviour and academic performance.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.
- Scikit-learn Documentation. <https://scikit-learn.org/stable/documentation.html>
- Pandas Documentation. <https://pandas.pydata.org/docs/>
- Seaborn Documentation. <https://seaborn.pydata.org/>